

Training Neural Networks

Loss Functions, `compile` and `fit`, and Choosing Activation Functions

Rokan Uddin Faruqui
Professor, Department of CSE

University of Chittagong
rufaruqui@cu.ac.bd

Week 5 · Part 1

From inference to training

Week 4 gave us

$$\mathbf{A}^{[l]} = g\left(\mathbf{A}^{[l-1]}\mathbf{W}^{[l]} + \mathbf{b}^{[l]}\right), \quad \mathbf{A}^{[0]} = \mathbf{X}$$

forward propagation with weights that someone else supplied.

Two questions were left open. Where do good values of $\mathbf{W}$ and $\mathbf{b}$ come from? And why should every unit use the sigmoid? Today answers both: training is the Week 3 recipe applied to the whole network, and g is a design choice with a small set of good defaults.

Today

1. Training in TensorFlow: `compile` and `fit`
2. Training details: model, loss, minimisation
3. Alternatives to the sigmoid: ReLU and linear
4. Choosing an activation for each layer
5. Why a network needs non-linear activations at all

Part 2 extends the output layer to many classes (softmax), replaces plain gradient descent with Adam, and opens up back-propagation.

TensorFlow Implementation

Three statements train a network

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.losses import BinaryCrossentropy

model = Sequential([                          # 1
    Dense(units=25, activation='sigmoid'),
    Dense(units=15, activation='sigmoid'),
    Dense(units=1,  activation='sigmoid')])

model.compile(loss=BinaryCrossentropy())      # 2

model.fit(X, y, epochs=100)                   # 3
```

The digit network of Week 4 ($64 \to 25 \to 15 \to 1$), now trained on `X` of shape $(m, 64)$ and labels `y` of shape $(m,)$.

What each statement does

1. **Specify the model**: architecture and activations, so TensorFlow knows how to compute $f(\mathbf{x})$.
2. **Specify the loss**: how wrong one prediction is.
3. **Minimise the cost**: run gradient descent for `epochs` passes over the training set.

Epoch

One full pass through all m training examples. `epochs=100` means the optimiser sees every example 100 times.

Training Details

The same three steps as logistic regression

Step	Logistic regression (Week 3)	Neural network (today)
1. Model: how to compute $f(\mathbf{x})$	$f_{\mathbf{w},b}(\mathbf{x}) = g(\mathbf{w} \cdot \mathbf{x} + b)$	`Sequential([Dense(...), ...])`
2. Loss and cost	$L = -y \log f - (1-y)\log(1-f)$ $J(\mathbf{w}, b) = \frac{1}{m}\sum_i L\left(f(x^{(i)}), y^{(i)}\right)$	`model.compile(loss=BinaryCrossentropy())`
3. Minimise J	repeat: $w_j \leftarrow w_j - \alpha\, \partial J/\partial w_j$, $b \leftarrow b - \alpha\, \partial J/\partial b$	`model.fit(X, y, epochs=100)`

Nothing new in principle

A network is a more flexible f. The loss, the cost, and gradient descent are the ones you already implemented by hand in the Week 3 lab.

What *is* new

J now depends on every $\mathbf{W}^{[l]}$ and $\mathbf{b}^{[l]}$ — for the digit network, 2031 parameters. Computing all 2031 partial derivatives efficiently is the job of **back-propagation** (Part 2).

Step 2: the loss, and the cost

Binary cross-entropy

For a label $y \in \{0, 1\}$ and output $f(\mathbf{x}) \in (0, 1)$:

$$L\big(f(\mathbf{x}), y\big) = -y \log f(\mathbf{x}) - (1 - y) \log\big(1 - f(\mathbf{x})\big)$$

The name comes from information theory; it is exactly the logistic loss of Week 3.

The cost averages the loss over the training set:

$$J\big(\mathbf{W}, \mathbf{B}\big) = \frac{1}{m} \sum_{i=1}^{m} L\big(f(x^{(i)}), y^{(i)}\big)$$

where $\mathbf{W}, \mathbf{B}$ stand for all weights and biases of all layers.

Worked numbers (Week 4 network)

$\mathbf{x}$	$f(\mathbf{x})$	L if $y = 1$	L if $y = 0$
$[1, 2]$	0.8094	0.2115	1.6576
$[-1, -1]$	0.0562	2.8788	0.0578

Confident and right: small loss. Confident and wrong: large loss.

Regression instead?

To predict a number, use the squared error:

$$L = \tfrac{1}{2}\big(f(\mathbf{x}) - y\big)^2$$

```
model.compile(loss=MeanSquaredError())
```

Step 3: minimise the cost

Gradient descent, for every layer

repeat $\{$

$$\mathbf{W}^{[l]} \leftarrow \mathbf{W}^{[l]} - \alpha \frac{\partial J}{\partial \mathbf{W}^{[l]}}, \qquad \mathbf{b}^{[l]} \leftarrow \mathbf{b}^{[l]} - \alpha \frac{\partial J}{\partial \mathbf{b}^{[l]}}$$

for $l = 1, \ldots, L$ $\}$

`fit` does exactly this. It computes the partial derivatives by **back-propagation**, and by default uses a better update rule than the plain one above (Adam, Part 2).

Libraries do the calculus

In 2010 a researcher derived and coded every gradient by hand. Today TensorFlow and PyTorch differentiate any network you can write — the library builds the derivatives from the forward computation.

Mature libraries, but understand them

Use the library the way you use a sort routine — but, as with sorting, know what it computes, its cost, and how it can fail. When training diverges or stalls, the diagnosis needs Weeks 1–3, not a different library.

```
history = model.fit(X, y, epochs=100)
history.history['loss']      # J per epoch
```

Plot this curve; it is the learning curve of Week 2.

Alternatives to the Sigmoid

Why not sigmoid everywhere?

Demand prediction, revisited

One hidden unit was meant to estimate *awareness* of a product. The sigmoid forces $0 < a < 1$: barely aware, or fully aware. But awareness can grow without limit — a product can go viral.

What we want for such a unit: 0 when the input is negative, and free to grow without bound otherwise. That function is

$$g(z) = \max(0, z).$$

ReLU — rectified linear unit

$$g(z) = \max(0, z)$$

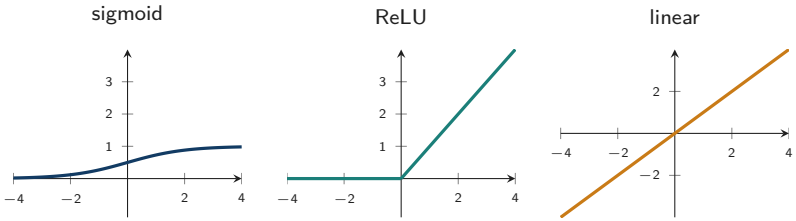
Zero for $z < 0$; the identity for $z \geq 0$. The name is historical; just say ReLU.

Linear activation

$$g(z) = z$$

Often described as "no activation". A unit with it computes $\mathbf{w} \cdot \mathbf{x} + b$ — linear regression.

The three activations you will use most



	$g(z)$	range	where it is used
sigmoid	$1/(1+e^{-z})$	$(0,1)$	output of a binary classifier
ReLU	$\max(0,z)$	$[0,\infty)$	hidden layers (default); non-negative outputs
linear	z	$(-\infty,\infty)$	output of a regression with any sign

Others exist — tanh, leaky ReLU, swish — and occasionally help; these three cover most needs. Part 2 adds softmax.

Worked example: ReLU in the hidden layer

Setting

The $2 \to 3 \to 1$ network of Week 4, with the *same* weights, but the hidden layer now uses ReLU. The output stays sigmoid.

$$\mathbf{W}^{[1]} = \begin{bmatrix} 1 & -3 & 5 \\ 2 & 4 & -6 \end{bmatrix}, \quad \mathbf{b}^{[1]} = [-1, 1, 2]$$

$$\mathbf{w}^{[2]} = [2, -1, -3], \quad b^{[2]} = 0.5$$

Key idea

$\mathbf{z}^{[1]}$ is unchanged — only g changed. ReLU zeros out the negative entries and passes the rest through *unsquashed*.

	$\mathbf{x} = [1, 2]$	$\mathbf{x} = [-1, -1]$
$\mathbf{z}^{[1]}$	$[4,\ 6,\ -5]$	$[-4,\ 0,\ 3]$
$\mathbf{a}^{[1]}$, sigmoid (Wk 4)	$[0.982,\ 0.998,\ 0.007]$	$[0.018,\ 0.5,\ 0.953]$
$\mathbf{a}^{[1]}$, ReLU	$[4,\ 6,\ 0]$	$[0,\ 0,\ 3]$
$z^{[2]}$	$8 - 6 - 0 + 0.5 = 2.5$	$0 - 0 - 9 + 0.5 = -8.5$
$a^{[2]} = g(z^{[2]})$	$\mathbf{0.9241}$	$\mathbf{0.0002}$

Same predictions as Week 4 ($\hat{y} = 1$, then $\hat{y} = 0$), but more confident: the hidden activations are no longer capped at 1.

A dead unit

If a ReLU unit's z is negative for *every* training example, it outputs 0 always, and its gradient is 0: it never learns. Unit 3 above is off for the first input — harmless as long as some inputs switch it on.

Choosing Activation Functions

The output layer: let the target decide

Target y	output g	loss	example
0 or 1	sigmoid	binary cross-entropy	spam or not
any real number	linear	mean squared error	tomorrow's change in a share price
non-negative number	ReLU	mean squared error	house price, rainfall
one of N classes	softmax	sparse categorical cross-entropy	digit 0–9 (Part 2)

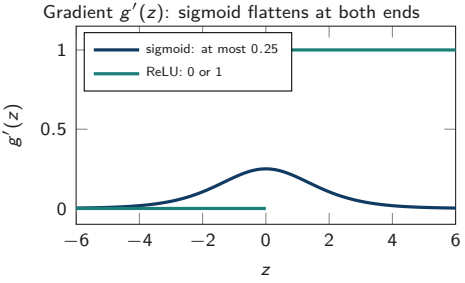
Match the range

Choose the output activation whose range is the set of values y can take. Then choose the loss that goes with it — the two come as a pair.

Mismatch

A sigmoid output with house prices as targets can never exceed 1: the loss stays huge and no amount of training fixes it. A linear output with binary cross-entropy can produce $f \leq 0$, where $\log f$ is undefined.

Hidden layers: ReLU by default



Why ReLU wins in hidden layers

- **Cheaper**: a comparison, not an exponential.
- **Flat on one side only.** The sigmoid is flat for large $|z|$ in both directions, so its gradient is tiny there and gradient descent crawls. ReLU's gradient is 1 for every $z > 0$.

In practice networks with ReLU hidden layers train noticeably faster.

```
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'),
    Dense(units=1,  activation='sigmoid')])
```

Why Do We Need Activation Functions?

A network of linear units is one linear unit

The algebra

Let every layer use $g(z) = z$. Two layers give

$$\mathbf{a}^{[1]} = \mathbf{x}\mathbf{W}^{[1]} + \mathbf{b}^{[1]}$$

$$\mathbf{a}^{[2]} = \mathbf{a}^{[1]}\mathbf{W}^{[2]} + \mathbf{b}^{[2]}$$

$$= \mathbf{x}\underbrace{\mathbf{W}^{[1]}\mathbf{W}^{[2]}}_{\mathbf{W}'} + \underbrace{\mathbf{b}^{[1]}\mathbf{W}^{[2]} + \mathbf{b}^{[2]}}_{\mathbf{b}'}$$

— a single linear layer with parameters $\mathbf{W}', \mathbf{b}'$. By induction, the same holds for any number of layers.

Our network, all linear

$\mathbf{W}' = \mathbf{W}^{[1]}\mathbf{w}^{[2]\top} = [-10,\ 18]^\top$,
$b' = \mathbf{b}^{[1]} \cdot \mathbf{w}^{[2]} + b^{[2]} = -8.5$.

$\mathbf{x}$	two layers	$-10x_1 + 18x_2 - 8.5$
$[1, 2]$	17.5	17.5
$[-1, -1]$	-16.5	-16.5

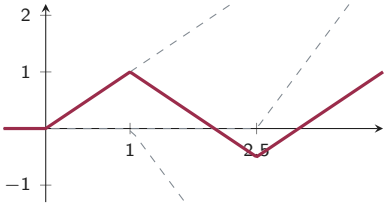
Consequences

- Linear hidden layers + linear output = linear regression.
- Linear hidden layers + sigmoid output = logistic regression.

Depth buys nothing without a non-linear g.

What the non-linearity buys

Sum of shifted ReLUs: $f(x) = r(x) - 2\,r(x-1) + 2\,r(x-2.5)$



Piecewise-linear building blocks

Each ReLU unit contributes one "hinge" at the point where its z crosses 0. The next layer adds hinges with weights: the slope is 1 on $(0, 1)$, $1 - 2 = -1$ on $(1, 2.5)$, and $-1 + 2 = 1$ beyond 2.5.

Universal approximation

With enough hidden units, sums of hinges approximate any continuous function on a bounded region as closely as you like. "Enough" can be very large — which is why depth, data and training still matter.

Rule of thumb

Never use a linear activation in a hidden layer. Use ReLU there, and pick the output activation from the target.

Wrap-up and Assessment

Summary

What to remember

- Training = specify the model, specify the loss, minimise the cost — `Sequential`, `compile`, `fit`.
- Binary cross-entropy is the logistic loss; mean squared error for regression.
- `fit` runs gradient descent; back-propagation supplies the gradients.
- ReLU $\max(0, z)$ in hidden layers; output activation chosen from the range of y.
- Without a non-linear activation, any depth collapses to one linear layer.

Formulas

$$L = -y \log f - (1 - y) \log(1 - f)$$

$$\mathrm{ReLU}(z) = \max(0, z), \quad \mathrm{linear}(z) = z$$

$$\mathbf{W}' = \mathbf{W}^{[1]}\mathbf{W}^{[2]}, \quad \mathbf{b}' = \mathbf{b}^{[1]}\mathbf{W}^{[2]} + \mathbf{b}^{[2]}$$

Next

Part 2: multiclass classification and softmax, a numerically safer implementation, the Adam optimiser, convolutional layers, and back-propagation.

Descriptive questions

1. List the three steps of training a model. For each, give the logistic-regression version and the TensorFlow statement that performs it. **[6]**
2. Define an epoch. If $m = 60\,000$ and `epochs=20`, how many times is each example seen during training? **[4]**
3. Write the binary cross-entropy loss. Compute it for $f(\mathbf{x}) = 0.9$ when $y = 1$ and when $y = 0$, and explain the difference. **[6]**
4. Define ReLU and the linear activation, sketch both, and state their ranges. **[5]**
5. For each target, choose the output activation and loss, with a reason: (a) whether a loan defaults; (b) tomorrow's temperature in °C; (c) the number of visitors to a website. **[6]**
6. Give two reasons ReLU is preferred over the sigmoid in hidden layers. **[5]**

Descriptive questions (continued)

7. Prove that a two-layer network whose layers both use $g(z) = z$ computes a linear function of $\mathbf{x}$. Give $\mathbf{W}'$ and $\mathbf{b}'$ in terms of the layers' parameters. **[8]**
8. A network has linear hidden layers and a sigmoid output. Which model from Module 1 is it equivalent to, and why? **[5]**
9. Using the worked-example weights with a ReLU hidden layer, compute $a^{[2]}$ for $\mathbf{x} = [0, 0]$ and for $\mathbf{x} = [2, 0]$. **[8]**
10. What is a "dead" ReLU unit? Why does gradient descent not revive it? **[5]**
11. Sketch $f(x) = r(x) - r(x - 2)$, where r is ReLU. Which familiar shape is it, and how many hidden units does it need? **[6]**
12. Explain why `model.fit` can train a network without you writing any derivative, and what you must still understand to debug it. **[5]**

Design questions

1. **Design a rainfall model.** The Bangladesh Meteorological Department wants tomorrow's rainfall (mm) at Chattogram from today's readings at 40 stations. Choose the architecture, every layer's activation, and the loss. Explain what would go wrong with a sigmoid output, and with a linear output. **[12]**
2. **Design an experiment: sigmoid vs. ReLU.** Design a fair experiment that tests the claim "ReLU hidden layers train faster than sigmoid ones" on the digit data. Specify what you hold fixed, what you measure, and how many runs you need. **[12]**
3. **Design a test for a collapsed network.** A colleague's 5-layer model performs exactly as well as logistic regression. Design a diagnostic — using the model's weights and a few inputs — that tests whether its hidden layers are accidentally linear. **[10]**
4. **Design a piecewise-linear fit.** Using ReLU hidden units and a linear output, design (by hand) a network that outputs 0 for $x < 0$, rises with slope 2 up to $x = 1$, and is flat at 2 after that. Give every weight. **[10]**

Reading

Primary text

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

→ **§10.1** — activation functions, ReLU, and why the non-linearity is essential; **§10.7** — fitting a network by gradient descent.

Supplementary

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

→ **§5.2** — network training: matching the output activation and error function to the target, and gradient-based optimisation.

Lab (CSE 816, Week 5): ReLU activation; softmax; multiclass; practice lab on neural networks for multiclass classification.

Practice quizzes: *Neural Network Training*; *Activation Functions*.